

译最终还是会还给事实一个公道。举个例子，如果稍后的源码内含这个：

```
LoggingMsgSender<CompanyZ> zMsgSender;  
MsgInfo msgData;  
...                               //在 msgData 内放置信息。  
zMsgSender.sendClearMsg(msgData); //错误！无法通过编译。
```

其中对 `sendClearMsg` 的调用动作将无法通过编译，因为在那个点上，编译器知道 **base class** 是个 **template** 特化版本 `MsgSender<CompanyZ>`，而且它们知道那个 **class** 不提供 `sendClear` 函数，而后者却是 `sendClearMsg` 尝试调用的函数。

根本而言，本条款探讨的是，面对“指涉 **base class members**”之无效 **references**，编译器的诊断时间可能发生在早期（当解析 **derived class template** 的定义式时），也可能发生在晚期（当那些 **templates** 被特定之 **template** 实参具现化时）。C++ 的政策是宁愿较早诊断，这就是为什么“当 **base classes** 从 **templates** 中被具现化时”它假设它对那些 **base classes** 的内容毫无所悉的缘故。

请记住

- 可在 **derived class templates** 内通过 `"this->"` 指涉 **base class templates** 内的成员名称，或藉由一个明白写出的“**base class** 资格修饰符”完成。

条款 44：将与参数无关的代码抽离 **templates**

Factor parameter-independent code out of templates.

Templates 是节省时间和避免代码重复的一个奇方妙法。不再需要键入 20 个类似的 **classes** 而每一个带有 15 个成员函数，你只需键入一个 **class template**，留给编译器去具现化那 20 个你需要的相关 **classes** 和 300 个函数。（**class templates** 的成员函数只有在被使用时才被暗中具现化，所以只有在这 300 个函数的每一个都被使用，你才会获得这 300 个函数。）**Function templates** 有类似的诉求。替换写许多函数，你只需要写一个 **function template**，然后让编译器做剩余的事情。技术是不是很高伟大，呵呵。

是的，唔……有时候啦。如果你不小心，使用 **templates** 可能会导致代码膨胀（*code bloat*）：其二进制码带着重复（或几乎重复）的代码、数据，或两者。其结果有可能源码看起来合身而整齐，但目标码（**object code**）却不是那么回事。肥胖不结实很难被视为时尚，所以你需要知道如何避免这样的二进制浮夸。

你的主要工具有个气势恢宏的名称：共性与变性分析（*commonality and variability analysis*），但其概念十分平民化。纵使你从未写过 **template**，你始

终做着这样的分析。

当你编写某个函数,而你明白其中某些部分的实现码和另一个函数的实现码实质相同,你会很单纯地重复这些码吗?当然不。你会抽出两个函数的共同部分,把它们放进第三个函数中,然后令原先两个函数调用这个新函数。也就是说,你分析了两个函数,找出共同的部分和变化的部分,把共同部分搬到一个新函数去,保留变化的部分在原函数中不动。同样道理,如果你正在编写某个 class,而你明白其中某些部分和另一个 class 的某些部分相同,你也不会重复这共同的部分。取而代之的是你会把共同部分搬移到新 class 去,然后使用继承或复合(见条款 32,38,39),令原先的 classes 取用这共同特性。而原 classes 的互异部分(变异部分)仍然留在原位置不动。

编写 templates 时,也是做相同的分析,以相同的方式避免重复,但其中有个窍门。在 non-template 代码中,重复十分明确:你可以“看”到两个函数或两个 classes 之间有所重复。然而在 template 代码中,重复是隐晦的:毕竟只存在一份 template 源码,所以你必须训练自己去感受当 template 被具现化多次时可能发生的重复。

举个例子,假设你想为固定尺寸的正方矩阵编写一个 template。该矩阵的性质之一是支持逆矩阵运算(matrix inversion)。

```
template<typename T,          //template 支持 n x n 矩阵,元素是
        std::size_t n>        //类型为T的 objects; 见以下
class SquareMatrix {          //关于 size_t 参数的信息
public:
    ...
    void invert( );            //求逆矩阵
};
```

这个 template 接受一个类型参数 T,除此之外还接受一个类型为 size_t 的参数,那是个非类型参数(non-type parameter)。这种参数和类型参数比起来较不常见,但它们完全合法,而且就像本例一样,相当自然。

现在,考虑这些代码:

```
SquareMatrix<double,5> sm1;
...
sm1.invert( );                //调用 SquareMatrix<double,5>::invert
SquareMatrix<double,10> sm2;
...
sm2.invert( );                //调用 SquareMatrix<double,10>::invert
```

这会具现化两份 `invert`。这些函数并非完全全相同，因为其中一个操作的是 5×5 矩阵而另一个操作的是 10×10 矩阵，但除了常量 5 和 10，两个函数的其他部分完全相同。这是 `template` 引出代码膨胀的一个典型例子。

如果你看到两个函数完全相同，只除了一个使用 5 而另一个使用 10，你会怎么做？你的本能会为它们建立一个带数值参数的函数，然后以 5 和 10 来调用这个带参数的函数，而不重复代码。你的本能很好，下面是对 `SquareMatrix` 的第一次修改：

```
template<typename T>                                //与尺寸无关的 base class,
class SquareMatrixBase {                             //用于正方形矩阵
protected:
    ...
    void invert(std::size_t matrixSize);             //以给定的尺寸求逆矩阵
    ...
};

template<typename T, std::size_t n>
class SquareMatrix: private SquareMatrixBase<T> {
private:
    using SquareMatrixBase<T>::invert;               //避免遮掩 base 版的
                                                    //invert; 见条款 33

public:
    ...
    void invert( ) { this->invert(n); }               //制造一个 inline 调用，调用
                                                    //base class 版的 invert。稍后
};                                                    //说明为什么这儿出现 this->
```

就如你所看到，带参数的 `invert` 位于 `base class SquareMatrixBase` 中。和 `SquareMatrix` 一样，`SquareMatrixBase` 也是个 `template`，不同的是它只对“矩阵元素对象的类型”参数化，不对矩阵的尺寸参数化。因此对于某给定之元素对象类型，所有矩阵共享同一个（也是唯一一个）`SquareMatrixBase class`。它们也将因此共享这唯一一个 `class` 内的 `invert`。

`SquareMatrixBase::invert` 只是企图成为“避免 `derived classes` 代码重复”的一种方法，所以它以 `protected` 替换 `public`。调用它而造成的额外成本应该是 0，因为 `derived classes` 的 `inverts` 调用 `base class` 版本时用的是 `inline` 调用（这里的 `inline` 是隐晦的，见条款 30）。这些函数使用 `"this->"` 记号，因为若不这样做，便如条款 43 所说，模板化基类（`templated base classes`，例如 `SquareMatrixBase<T>`）内的函数名称会被 `derived classes` 掩盖。也请注意 `SquareMatrix` 和 `SquareMatrixBase` 之间的继承关系是 `private`。这反应一个事实：

这里的 `base class` 只是为了帮助 `derived classes` 实现, 不是为了表现 `SquareMatrix` 和 `SquareMatrixBase` 之间的 **is-a** 关系 (关于 `private` 继承, 见条款 39)。

目前为止一切都好, 但还有一些棘手的题目没有解决。`SquareMatrixBase::invert` 如何知道该操作什么数据? 虽然它从参数中知道矩阵尺寸, 但它如何知道哪个特定矩阵的数据在哪儿? 想必只有 `derived class` 知道。`Derived class` 如何联络其 `base class` 做逆运算动作? 一个可能的做法是为 `SquareMatrixBase::invert` 添加另一个参数, 也许是个指针, 指向一块用来放置矩阵数据的内存起始点。那行得通, 但十之八九 `invert` 不是唯一一个可写为“形式与尺寸无关并可移至 `SquareMatrixBase` 内”的 `SquareMatrix` 函数。如果有若干这样的函数, 我们唯一要做的就是找出保存矩阵元素值的那块内存。我们可以对所有这样的函数添加一个额外参数, 却得一次又一次地告诉 `SquareMatrixBase` 相同的信息, 这样似乎不好。

另一个办法是令 `SquareMatrixBase` 贮存一个指针, 指向矩阵数值所在的内存。而只要它存储了那些东西, 也就可能存储矩阵尺寸。成果看起来像这样:

```
template<typename T>
class SquareMatrixBase {
protected:
    SquareMatrixBase(std::size_t n, T* pMem)           //存储矩阵大小和一个
    : size(n), pData(pMem) { }                       //指针, 指向矩阵数值。
    void setDataPtr(T* ptr) { pData = ptr; }          //重新赋值给 pData。
    ...
private:
    std::size_t size;                                //矩阵的大小。
    T* pData;                                         //指针, 指向矩阵内容。
};
```

这允许 `derived classes` 决定内存分配方式。某些实现版本也许会决定将矩阵数据存储在 `SquareMatrix` 对象内部:

```
template<typename T, std::size_t n>
class SquareMatrix: private SquareMatrixBase<T> {
public:
    SquareMatrix( )                                //送出矩阵大小和
    : SquareMatrixBase<T>(n, data) { }             //数据指针给 base class。
    ...
private:
    T data[n*n];
};
```

这种类型的对象不需要动态分配内存，但对象自身可能非常大。另一种做法是把每一个矩阵的数据放进 **heap**（也就是通过 **new** 来分配内存）：

```
template<typename T, std::size_t n>
class SquareMatrix: private SquareMatrixBase<T> {
public:
    SquareMatrix()                //将 base class 的数据指针设为 null,
        : SquareMatrixBase<T>(n, 0),    //为矩阵内容分配内存,
        pData(new T[n*n])           //将指向该内存的指针存储起来,
        { this->setDataPtr(pData.get()); } //然后将它的一个副本交给 base class
    ...
private:
    boost::scoped_array<T> pData;      //关于 boost::scoped_array,
};                                     //见条款 13.
```

不论数据存储于何处，从膨胀的角度检讨之，关键是现在许多——说不定是所有——**SquareMatrix** 成员函数可以单纯地以 **inline** 方式调用 **base class** 版本，后者由“持有同型元素”（不论矩阵大小）之所有矩阵共享。在此同时，不同大小的 **SquareMatrix** 对象有着不同的类型，所以即使（例如 **SquareMatrix<double,5>** 和 **SquareMatrix<double,10>**）对象使用相同的 **SquareMatrixBase<double>** 成员函数，我们也没机会传递一个 **SquareMatrix<double,5>** 对象到一个期望获得 **SquareMatrix<double,10>** 的函数去。很棒，对吗？

是的，很棒，但必须付出代价。硬是绑着矩阵尺寸的那个 **invert** 版本，有可能生成比共享版本（其中尺寸乃以函数参数传递或存储在对象内）更佳的代码。例如在尺寸专属版中，尺寸是个编译期常量，因此可以藉由常量的广传达到最优化，包括把它们折进被生成指令中成为直接操作数。这在“与尺寸无关”的版本中是无法办到的。

从另一个角度看，不同大小的矩阵只拥有单一版本的 **invert**，可减少执行文件大小，也就因此降低程序的 **working set**（译注：见下说明）大小，并强化指令高速缓存区内的引用集中化（**locality of reference**）。这些都可能使程序执行得更快速，超越“尺寸专属版”**invert** 的最优化效果。哪一个影响占主要地位？欲知答案，唯一的办法是两者都尝试并观察你的平台的行为以及面对代表性数据组时的行为。

译注：所谓 **working set** 是指对一个在“虚内存环境”下执行的进程（**process**）而言，其所使用的那一组内存页（**pages**）。

另一个效能评比所关心的主题是对象大小。如果你不介意，可将前述“与矩阵大小无关的函数版本”搬至 **base class** 内，这会增加每一个对象的大小。例如在我

刚才展示的例子中，每一个 `SquareMatrix` 对象都有一个指针指向 `SquareMatrixBase` class 内的数据。虽然每个 `derived class` 已经有一种取得数据的办法，这会对每一个 `SquareMatrix` 对象增加至少一个指针那么大。当然也可以修改设计，拿掉这些指针，但是再一次，这其中需要若干取舍。例如令 `base class` 贮存一个 `protected` 指针指向矩阵数据，会导致丧失封装性，如条款 22 所言。也可能导致资源管理上的混乱和复杂；是的，如果 `base class` 存储一个指针指向矩阵数据，那些数据空间也许是动态分配获得，也许存储于 `derived class` 对象内（如稍早所见），如何判断这个指针该不该被删除呢？这样的问题有其答案，但你愈是尝试精密的做法，事情变得愈是复杂。从某个角度看，一点点代码重复反倒看起来有点幸运了。

这个条款只讨论由 `non-type template parameters`（非类型模板参数）带来的膨胀，其实 `type parameters`（类型参数）也会导致膨胀。例如在许多平台上 `int` 和 `long` 有相同的二进制表述，所以像 `vector<int>` 和 `vector<long>` 的成员函数有可能完全相同——这正是膨胀的最佳定义。某些连接器（`linkers`）会合并完全相同的函数实现码，但有些不会，后者意味某些 `templates` 被具现化为 `int` 和 `long` 两个版本，并因此造成代码膨胀（在某些环境下）。类似情况，在大多数平台上，所有指针类型都有相同的二进制表述，因此凡 `templates` 持有指针者（例如 `list<int*>`, `list<const int*>`, `list<SquareMatrix<long, 3>*>` 等等）往往应该对每一个成员函数使用唯一一份底层实现。这很具代表性地意味，如果你实现某些成员函数而它们操作强型指针（*strongly typed pointers*，即 `T*`），你应该令它们调用另一个操作无类型指针（*untyped pointers*，即 `void*`）的函数，由后者完成实际工作。某些 C++ 标准程序库实现版本的确为 `vector`, `deque` 和 `list` 等 `templates` 做了这件事。如果你关心你的 `templates` 可能出现代码膨胀，也许你会想让你的 `templates` 也做相同的事情。

请记住

- `Templates` 生成多个 `classes` 和多个函数，所以任何 `template` 代码都不该与某个造成膨胀的 `template` 参数产生相依关系。
- 因非类型模板参数（`non-type template parameters`）而造成的代码膨胀，往往可消除，做法是以函数参数或 `class` 成员变量替换 `template` 参数。
- 因类型参数（`type parameters`）而造成的代码膨胀，往往可降低，做法是让带有完全相同二进制表述（`binary representations`）的具现类型（`instantiation types`）共享实现码。

条款 45：运用成员函数模板接受所有兼容类型

Use member function templates to accept "all compatible types."

所谓智能指针 (*Smart pointers*) 是“行为像指针”的对象，并提供指针没有的机能。例如条款 13 曾经提及 `std::auto_ptr` 和 `tr1::shared_ptr` 如何能够被用来在正确时机自动删除 heap-based 资源。STL 容器的迭代器几乎总是智能指针；无疑你不会奢望使用 “++” 将一个内置指针从 linked list 的某个节点移到另一个节点，但这在 `list::iterators` 身上办得到。

真实指针做得很好的一件事是，支持隐式转换 (*implicit conversions*)。Derived class 指针可以隐式转换为 base class 指针，“指向 non-const 对象”的指针可以转换为“指向 const 对象”……等等。下面是可能发生于三层继承体系的一些转换：

```
class Top { ... };
class Middle: public Top { ... };
class Bottom: public Middle { ... };
Top* pt1 = new Middle;           //将 Middle* 转换为 Top*
Top* pt2 = new Bottom;           //将 Bottom* 转换为 Top*
const Top* pct2 = pt1;           //将 Top* 转换为 const Top*
```

但如果想在用户自定的智能指针中模拟上述转换，稍稍有点麻烦。我们希望通过下代码通过编译：

```
template<typename T>
class SmartPtr {
public:                             //智能指针通常
    explicit SmartPtr(T* realPtr); //以内置（原始）指针完成初始化
    ...
};

SmartPtr<Top> pt1 =                  //将 SmartPtr<Middle> 转换为
    SmartPtr<Middle>(new Middle);    // SmartPtr<Top>
SmartPtr<Top> pt2 =                  //将 SmartPtr<Bottom> 转换为
    SmartPtr<Bottom>(new Bottom);    // SmartPtr<Top>
SmartPtr<const Top> pct2 = pt1;      //将 SmartPtr<Top> 转换为
                                     // SmartPtr<const Top>
```

但是，同一个 `template` 的不同具现体 (*instantiations*) 之间并不存在什么与生俱来的固有关系（译注：这里意指如果以带有 base-derived 关系的 B、D 两类型分别具现化某个 `template`，产生出来的两个具现体并不带有 base-derived 关系），所以编译器视 `SmartPtr<Middle>` 和 `SmartPtr<Top>` 为完全不同的 classes，它们之间的关系并不比……唔……并不比 `vector<float>` 和 `Widget` 更密切，呵呵。为了获得我们希望获得的 `SmartPtr` classes 之间的转换能力，我们必须将它们明确地编写出来。